Data and Storage

Lesson 4 of 12 · Where AI data lives, how to read it fast, and how vectors work.

In this lesson → ① Object storage and egress ② Small files vs shards ③ Checkpoints ④ Lakehouse and versions ⑤ Vectors, measured ⑥ HNSW ⑦ Interview Q&A
Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 Object storage: a filing cabinet

S3, Google Cloud Storage, Azure Blob: every file has a name; almost unlimited, cheap, very durable. You fetch **whole objects**; you do not edit the middle.

2 Region is truth, zone is speed



region = truth, zone = speed

Tier	Put here
regional standard (S3 Standard ~ $0.023 / GB-month list)	datasets, shards, final checkpoints, weights
one-zone fast (S3 Express, GCS Rapid)	hot shards, scratch, recent checkpoints (**never the only copy**)
cool / archive	old checkpoints, raw crawls, compliance copies

3 Egress is the real bill

> Storing is cheap. **Egress** (data leaving a cloud) and cross-zone / cross-region traffic are what cost. List prices (~2026, notebook): AWS ~$0.09/GB, Azure ~$0.087, GCP Premium ~$0.12; Cloudflare R2: $0.

Data out	GB	At $0.09 / GB (list)
1 TB	1,000	$90
10 TB	10,000	$900
100 TB	100,000	**$9,000**

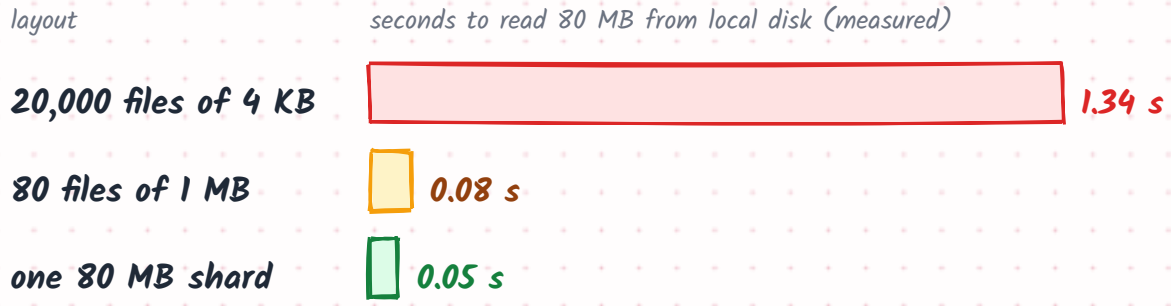
Arithmetic, l04_checkpoint_math.py. Volume tiers are lower; budget it before chasing cheap GPUs in another cloud. Move the job to the data, not the data to the job.

4 Small files: the beginner mistake

Object stores love big sequential reads. Every request has a fixed delay, so thousands of tiny reads pay that delay thousands of times. The cure: **shards**.

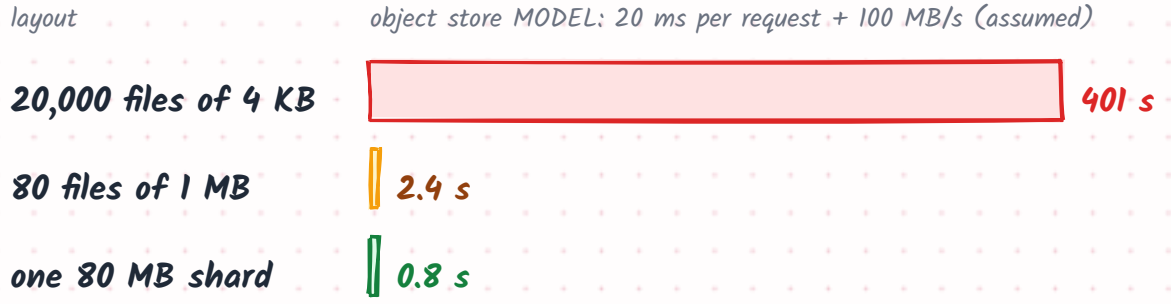
5 Experiment: 80 MB, three layouts

Layout	Seconds	MB/s	vs one file
20,000 files of 4 KB	1.337	60	25.1x
80 files of 1 MB	0.079	1,014	1.5x
one 80 MB shard	0.053	1,499	1.0x



MEASURED on local disk, page cache dropped before each read, best of three. Second run: 1.425 s, 0.069 s, 0.054 s. Script: l04_small_files.py.

Object store MODEL (assumed 20 ms per request, 100 MB/s per stream)



6 Shards and streaming

> A **shard** = 100 MB to 1 GB of samples read in order. Each worker holds about one shard plus a shuffle buffer.

- → **WebDataset**: plain .tar shards; a sample is sibling files with the same name
- → **Mosaic StreamingDataset (MDS)**: any cloud, resumable mid-epoch, same sample order on any GPU count
- → AWS: Mountpoint for S3 and the S3 Connector for PyTorch

7 When storage actually hurts

Dataset bytes = tokens x bytes per token. 1ST tokens as uint16 ids = **30 TB**, read about once per pass: text pretraining is rarely I/O-bound. Storage is stressed by **images and video, small random reads you cannot re-shard, and checkpoint bursts**. Profile before buying speed.

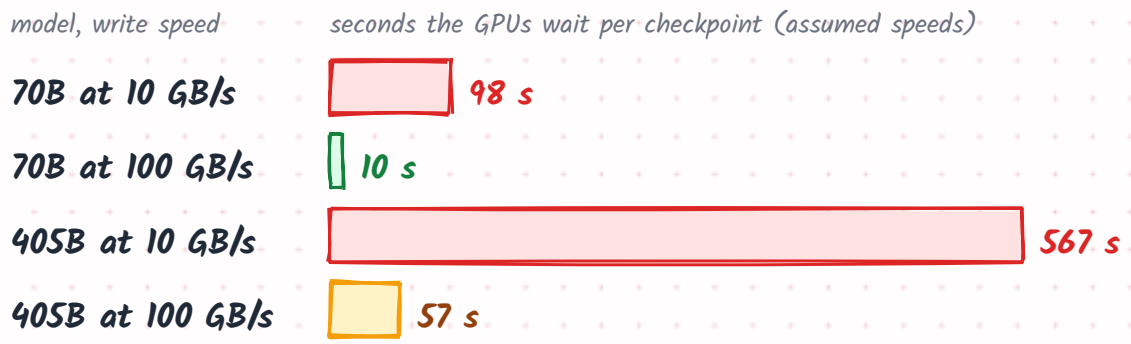
8 Fast file systems

- → Managed **Lustre** on AWS, GCP and Azure; WEKA, VAST, DDN at neoclouds
- → You buy bandwidth by buying capacity: throughput = TiB x tier (MB/s per TiB)
- → Pattern: bucket = master, fast file system = a job-sized copy, torn down after the run

9 Checkpoint size and the stall

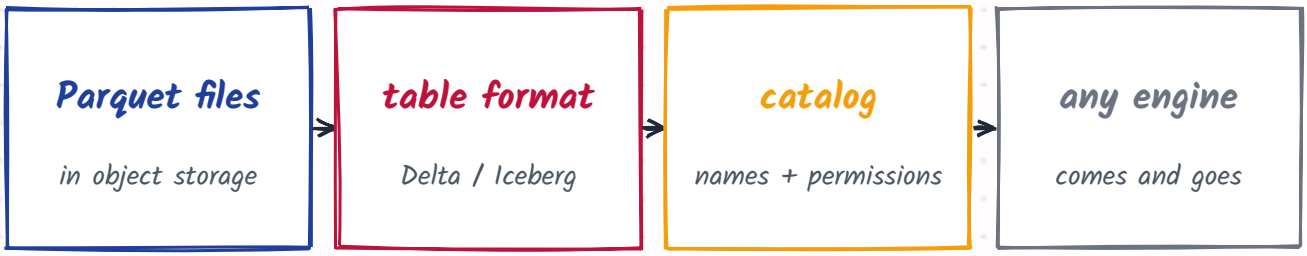
> **full checkpoint = 14 bytes per parameter** = bf16 weights 2 + fp32 master 4 + Adam m 4 + Adam v 4

Model	Size	Wait at 10 GB/s	Wait at 100 GB/s
70B	0.98 TB	98 s	10 s
405B	5.67 TB	567 s	57 s



Arithmetic with ASSUMED write speeds. Fix: **asynchronous saving**: snapshot to RAM or local NVMe, keep training, drain to the bucket in the background.

10 The lakehouse



engines come and go, the tables stay

Parquet files in object storage + a table format (Delta, Iceberg, Hudi: transactions and snapshots) + a catalog (names, permissions, lineage). Lock-in has moved from the file format to the **catalog**. Indexes and online stores are projections: design them to be **rebuilt from source**.

11 Version your data



pin every training run to a version

The retention trap
Delta time travel needs old files. `deletedFileRetentionDuration` defaults to **7 days** (log: 30). Set both to your lookback, e.g. 90 days, or last month's training set is gone.

12 Reproducibility and quality gates

- ✓ Record: data snapshot id, code commit, tokenizer / preprocessing version, embedding-model version
- ✓ Check before writing training sets, features or indexes: schema, nulls, duplicates, personal data, eval contamination
- ✓ Keep a model registry entry for every run

13 Vectors: meaning as numbers

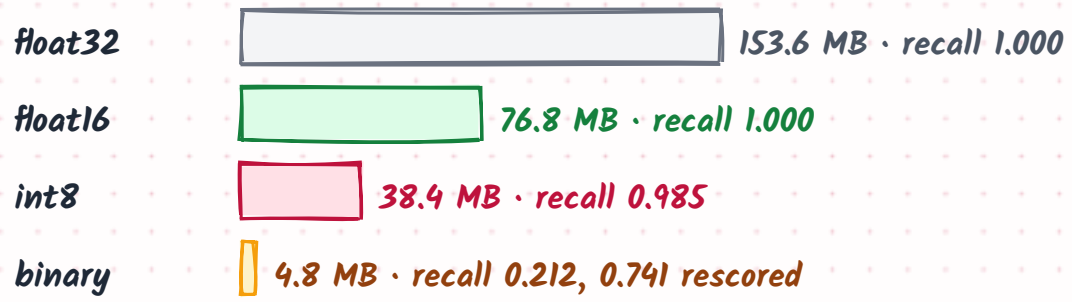
> An embedding model turns text or an image into a **vector**. Similar meaning = close numbers. Search = embed the question, find the nearest vectors. **Exact** search compares with all (slow at scale). **Approximate** (ANN) trades a little recall for big speed.

> **bytes per vector = dimensions x bytes per number** · 768 x 4 = 3,072 B · 50,000 vectors = 153.6 MB · 10M = 30.7 GB

14 Experiment: shrinking vectors

50,000 SYNTHETIC vectors, 768 numbers, 100 gaussian clusters (not real embeddings). 300 queries. Truth = exact float32 search. recall@10 = share of the true top 10 found. One CPU thread. Script: 104_vectors.py.

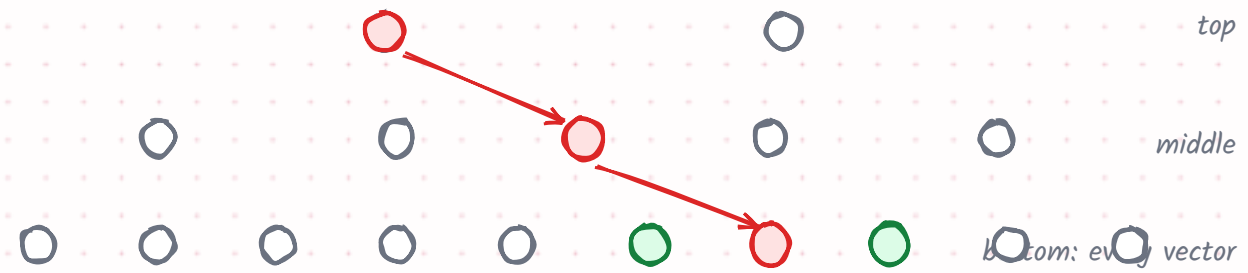
format — 50,000 vectors x 768 numbers (synthetic): size and recall@10



Format	Bytes / vector	Total	recall@10
float32	3,072	153.6 MB	1.000
float16	1,536	76.8 MB	**1.000**
int8	768	38.4 MB	**0.985**
binary (Hamming only)	96	4.8 MB	0.212
binary, top-100 rescored with float32	96	4.8 MB	0.741

Surprise
Plain bits are only a first filter. The notebook says binary compression is weaker below about 384 dimensions; rescoring recovered 0.21 to 0.74 here.

15 HNSW: motorways, roads, streets

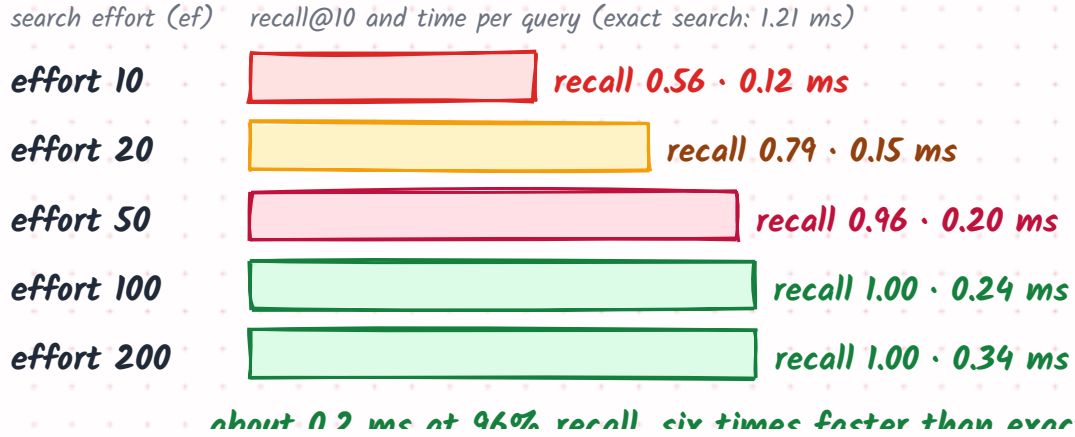


hop toward the query, drop a layer when stuck, then look around: the nearest friends

- Layered graph: top layers sparse, bottom layer holds every vector
- Search: enter at the top, hop to the nearest node, drop a layer when stuck, then look around on the bottom layer
- **M** = links per node (more RAM) · **ef** = how hard you look (slower, more recall)

> **HNSW RAM ≈ N x (4d + 8M) bytes** · 50,000 x (3,072 + 128) = **160 MB** · real index file: **161 MB**

16 HNSW: recall against effort



ef	recall@10	ms / query	vs exact (1.21 ms)
10	0.563	0.118	10x
20	0.787	0.147	8x
50	**0.963**	0.196	**6x**
100	0.998	0.241	5x
200	1.000	0.341	4x

Measured with hnswlib, M=16, efConstruction=100. Build 16.8 s. The gap to exact search grows with the collection.

17 Other index families

Family	Lives in	Good at
HNSW	RAM	low latency, high recall
IVF + PQ	RAM, compressed	small memory, fast build
DiskANN	SSD + PQ in RAM	billions per machine
Object-tiered (S3 Vectors)	cheap storage	cheapest, about 100 ms

18 Choosing a vector store

Situation	Start with
already on Postgres, millions of vectors	pgvector, halfvec (fp16), HNSW
huge and cheap, slower is fine	object-backed (S3 Vectors)
heavy, low-latency search	a dedicated vector database
any of them	add keyword search (lesson 8)

19 ShopNest Copilot: data plan

Data	Where
tickets, eval sets	regional object storage, in shards
checkpoints	async save, regional copy
support-article vectors	pgvector, halfvec, HNSW
index	rebuildable: record the embedding-model version

20 Words to keep

Word	Meaning
Shard	a big file packed with many samples
Egress	the charge for data leaving a cloud
Recall	share of true nearest neighbours returned
Quantisation	fewer bits per number to save memory

21 Interview questions

"Why are millions of small files slow on object storage?"

Every request pays a fixed delay, so tiny reads are dominated by it. In my test 20,000 files of 4 KB read 25.1x slower than one 80 MB shard. Pack samples into 100 MB to 1 GB shards (WebDataset, MDS).

Trap → Buying a faster file system before re-sharding the data.

"Where does a cloud data bill really come from?"

Egress and cross-zone or cross-region traffic, not storage. 100 TB out at a $0.09/GB list price is $9,000, while S3 Standard storage is about $0.023 per GB-month. Move the job to the data.

Trap → Chasing cheaper GPUs in another cloud and paying to move the dataset there.

"How large is a training checkpoint, and how do you avoid the stall?"

About 14 bytes per parameter: bf16 weights 2 + fp32 master 4 + Adam 4 + 4. 70B is about 0.98 TB: 98 s at 10 GB/s. Save asynchronously: snapshot to RAM or NVMe, keep training, drain to the bucket.

Trap → Keeping the only checkpoint copy in one-zone fast storage.

"How do you make a training set reproducible?"

Record the data snapshot id, code commit, tokenizer and embedding-model versions. With Delta, set retention to your lookback (for example 90 days): the default 7 days deletes last month's training set.

Trap → Assuming time travel keeps old data forever.

"How do you cut vector index memory?"

Use fewer bytes per number. float16 kept recall@10 at 1.000 with half the memory; int8 gave 0.985 at a quarter. Binary alone fell to 0.212; rescoring the top 100 with float32 recovered 0.741.

Trap → Using binary vectors without a rescoring step.

"Explain HNSW and its knobs."

A layered graph: sparse top layers, every vector on the bottom. M is links per node (more RAM), ef is how hard you search. In my test ef 50 gave 0.963 recall about 6x faster than exact search.

Trap → Tuning ef without measuring recall on your own data.

22 Interview corner

Q. Why not store training data as millions of small files?

A. Every object-store request has a fixed delay, so small reads are dominated by it. Pack samples into 100 MB to 1 GB shards. In my test small files read 25x slower.

Q. How big is a training checkpoint?

A. About 14 bytes per parameter: 70B is about 1 TB. Use asynchronous saving so GPUs do not wait.

Q. How do you cut vector memory?

A. float16 or int8: recall 1.000 and 0.985 at a half and a quarter of the memory. Plain bits drop to 0.21 unless you rescore.

Q. What do the HNSW knobs do?

A. M sets links per node (RAM), ef sets search effort (latency vs recall). In my test ef 50 gave 0.96 recall about 6x faster than exact.

23 Your turn

104_small_files.py, **104_vectors.py** (change dimensions), **104_checkpoint_math.py**.

⚡ Revise in 60 seconds

- → region = truth · zone = speed · egress = the bill
- → 20,000 small files: **25x** slower than one shard
- → checkpoint = **14 B** per parameter
- → Delta default retention: 7 days
- → 768-d float32 = 3,072 B per vector
- → int8: **0.985** recall at ¼ the memory
- → HNSW ef 50: **0.96** recall, 6x faster
- → HNSW RAM ≈ N x (4d + 8M)

Next lesson → CAI-05
Training at scale: where the memory goes and how to split a model
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**